

I would like to build a Mac native desktop app named Filer. It will initially have 2 tabs

- Indexing
- Filing

Eventually there will be a third tab

- Filing

The indexing tab will allow the selection of a root folder to index. The folder can be dragged into the tab to kick off the process, or a button can be pressed to pop up a

Show more ▾

I'll frame this as a practical architecture you can actually build: Mac-native packaging, Python AI/indexing backend, React UI, local-only storage, and a path from MVP to the chat/RAG assistant.

Thought for 9s >

I'd build **Filer** like this:

Recommended stack

Desktop shell: Tauri 2 + React + TypeScript

Tauri lets you use React while producing a lightweight native desktop app; it supports any frontend framework and integrates with system APIs. Tauri

Electron is easier but heavier because it bundles Chromium and Node.js. Electron

Backend: Python FastAPI sidecar process

Run a local-only FastAPI service on `127.0.0.1`, launched by Tauri. Python handles indexing, embeddings, RAG, file parsing, OCR, and future AI workflows.

Local datastore:

Use:

SQLite → file inventory, paths, metadata, jobs, move history
 LanceDB → vectors / semantic search
 SQLite FTS5 → keyword search
 File hashes → change detection / deduplication

I'd strongly consider **LanceDB over Chroma** for this app. Chroma is fine and stores/query embeddings locally, but LanceDB is an embedded local vector DB with Python support and local-path storage, which fits a no-sign-in desktop app well.

Chroma Docs [+2](#)

High-level architecture

```

Filer.app
├── Tauri shell
│   ├── React UI
│   ├── macOS file picker
│   ├── drag/drop handling
│   └── launches Python sidecar
├── Python backend
│   ├── FastAPI API
│   ├── indexing worker
│   ├── file parser pipeline
│   ├── embedding pipeline
│   ├── folder suggestion engine
│   └── future RAG assistant
└── Local app data
    ├── filer.db           SQLite metadata DB
    ├── fts.db            optional SQLite FTS index
    ├── lancedb/          vector index
    ├── thumbnails/
    └── logs/
  
```

Core data model

SQLite tables:

```

files
- id
- absolute_path
- filename
- extension
- mime_type
  
```

- size_bytes
- created_at
- modified_at
- content_hash
- indexed_at
- status

folders

- id
- absolute_path
- parent_path
- folder_name

documents

- file_id
- extracted_text
- parser_used
- extraction_status

chunks

- id
- file_id
- chunk_text
- chunk_index
- token_count

filing_actions

- id
- source_path
- destination_path
- accepted_suggestion
- moved_at

Vector DB rows:

chunk_id
file_id
path
folder_path
filename
chunk_text
embedding
metadata

Indexing flow

1. User selects or drags a root folder.

2. Backend walks the filesystem.
3. For each file:
 - capture metadata
 - hash file
 - skip unchanged files
 - extract text
 - chunk text
 - embed chunks
 - store vectors
 - update SQLite metadata
4. Optional: watch the folder with macOS filesystem events.

For parsing, use:

PDFs	→ pymupdf / pypdf
DOCX	→ python-docx
TXT/MD	→ native read
CSV/XLSX	→ pandas/openpyxl
Images	→ OCR later, probably optional
Email	→ later: eml/msg parser

Filing suggestion logic

For each inbox file, generate a destination score using several signals:

1. Semantic similarity
Compare inbox file embedding to folder/document embeddings.
2. Filename similarity
Match names like "Fidelity", "Eliquis", "mortgage", "Spring Hill".
3. Metadata similarity
Date, source folder, extension, document type.
4. Folder history
If user previously filed similar files into a folder, boost it.
5. Rules
User-defined rules later:
"Anything from Fidelity goes to Finance/Brokerage/Fidelity"

The UI should show:

Suggested folder: /Finance/Credit Cards/Fidelity
Confidence: 87%
Why: filename mentions Fidelity + content matches prior Fidelity statements
[Move here]

React UI

Initial tabs:

Indexing
Filing

Later third tab should probably be:

Assistant

You mentioned “Eventually there will be a third tab - Filing,” but I think you meant **Assistant**.

Indexing tab

[Drop root folder here]

or

[Choose Folder]

Indexed folders:
- /Users/eric/Documents
- /Users/eric/Downloads

Progress:
Scanning → Extracting → Embedding → Complete

Filing tab

Inbox	Folder tree
bank_statement.pdf	Documents
fidelity_card_terms.pdf	Finance
medical_bill.pdf	Credit Cards
	Fidelity ← highlighted

Actions:



Drag file → folder → move file
Undo last move

Future AI assistant

Use the same indexes:

User question
→ classify intent
→ retrieve relevant chunks from LanceDB + FTS
→ rerank
→ answer with citations to local files
→ optionally open source file

Example:

"What are the side effects of diltiazem?"

Answer:

Based on /Medical/Medications/diltiazem_info.pdf...

For local/no-sign-in mode, support:

Local embeddings: sentence-transformers / bge-small / nomic-embed-text
Local LLM: Ollama optional
Cloud LLM: optional user-provided API key later

My recommended MVP

Build in this order:

1. Tauri + React shell
2. Python FastAPI sidecar
3. Folder picker + drag/drop
4. SQLite file inventory
5. Text extraction for PDF, TXT, DOCX
6. LanceDB embeddings

7. Inbox pane + folder tree
8. Suggest destination folder
9. Move file + undo
10. Assistant tab later

Final recommendation

Use:

```
Tauri 2  
React + TypeScript  
Python FastAPI sidecar  
SQLite  
SQLite FTS5  
LanceDB  
sentence-transformers embeddings  
PyMuPDF / pypdf / python-docx
```

That gives you a real Mac desktop app, keeps your AI backend in Python, avoids sign-in, keeps user files local, and leaves a clean path toward RAG-based file Q&A.

     ...  Sources